# 01 / Workflow slide script

How one fix is judged, step by step | Approximately 90 seconds

Now that we have introduced PRATIRODH, this slide explains how we judge whether a proposed fix is ready for human review.

We begin by reading the project and protecting its tests and configuration. Then we establish a baseline: does normal functionality work before the repair? Next, we reproduce the vulnerability so we know exactly what needs fixing.

We obtain a candidate patch from a developer or our local AI model, apply it to a copy, and run security and functional checks. The repair must block the tested attacks while preserving legitimate behavior.

Then we deliberately weaken copies of the fix. These are called mutants. This checks whether our verification can actually detect an incomplete repair.

The bottom-left panel shows the verdict: failed requirements cause rejection; missing evidence means insufficient evidence; passing the required checks leads to ready for human review.

The middle panel shows the execution boundary: the controller manages the workflow, while restricted containers execute the checks. The separate audit-worker stage shown here remains future evaluation work.

Finally, the right-hand example compares an incomplete fix with a full fix. Both block one attack path, but the incomplete fix misses another. That is why simply passing ordinary tests is not enough.

Our final output is a candidate repair supported by signed evidence, not an automatically deployed patch. A human still reviews and decides.

**Transition: Let me now demonstrate that workflow using an actual local AI repair.**

Presenter reminder: Do not claim every step is sandboxed or every possible weakness is tested. Retry counts and resource limits depend on the configured workflow.

# 02 / Start the live demo

4-5 minutes of explanation, plus model execution time | http://127.0.0.1:8779/

## 1. Homepage

**DO: Start at the top, scroll briefly through the workflow, then click Open workspace.**

PRATIRODH is a security repair workbench. An AI-generated patch can look correct while leaving a vulnerability or breaking normal behavior. Our project generates a candidate repair, challenges it with verification checks, and preserves signed evidence for human review.

If showing a homepage example, identify it as a recorded demonstration, not the live run.

## 2. Workspace

**DO: Select SQL injection / CWE-89, AI repair, and pratirodh-qwen:7b-hf.**

For this demo, I am using a controlled Python example containing SQL injection. CWE-89 identifies this weakness category: user input can change the meaning of a database query.

The example is prepared beforehand, but the repair is generated live using Qwen2.5-Coder 7B through Ollama on this machine. The model proposes the patch; separate verification checks evaluate it.

## 3. Progress

**DO: Click Generate repair once. Wait for the outcome and open the resulting run.**

The application prepares the input, generates a candidate patch, and runs verification. Security checks test malicious inputs, while functional checks confirm legitimate behavior still works.

Do not promise success while it is running or start duplicate jobs. If asked: the demo uses the existing Qwen model. The experimental fine-tuned adapter was not promoted because evaluation found a security regression.

## 4. Overview

**DO: Point to the actual decision and read the displayed check counts.**

If the decision is READY_FOR_REVIEW: This candidate passed the configured verification checks and is ready for human review. That does not mean it has been deployed or proven secure against every possible attack.

If rejected, explain the displayed reason. If incomplete or failed, say execution did not establish an acceptable repair result. A rejection is a valid verification outcome.

# 03 / Inspect the evidence

Explain the code first, then explain why the verification supports the decision.

## 5. Patch

**DO: Open Patch and point to the actual query change.**

Here we can inspect what the model changed. The repaired query separates SQL instructions from user input using placeholders and separately supplied parameters. Input is treated as data rather than part of the SQL command.

If asked: in SQLite, a question mark is a value placeholder. Values are passed separately; (name,) is a one-item Python tuple. Describe the actual patch on screen rather than assuming its syntax.

## 6. Requests

**DO: Open Requests. Show one security check and one legitimate-behavior check.**

These results show how the candidate behaved under the tested inputs. We need both: blocking malicious behavior and preserving valid functionality.

## 7. Challenge

**DO: Open Challenge and read the actual mutant results.**

Passing tests alone is not enough if those tests are weak. We deliberately weaken copies of the repair, called mutants, and check whether verification detects them.

A confirmed-unsafe mutant is a weakened version shown to violate the tested security requirement. It refers to that altered copy, not the accepted candidate. This assesses the checks within the configured mutation set, not every possible attack.

## 8. Provenance and export

**DO: Show model identity, model-call count, signature and freshness. Click Download signed evidence.**

Provenance records how this result was produced. A verified signature protects the integrity of the evidence, while current freshness means it matches the required verification inputs.

The ZIP contains the report, supporting artifacts, integrity inventory and signature material. It can be independently verified against the trusted public key. Signed evidence makes tampering detectable; it does not guarantee absolute security.

Downloading is not itself independent verification. Only claim verification of this particular ZIP if you actually performed it. Keep the original download unchanged.

# 04 / Review and close

Record an honest human decision, then explain where the product boundary ends.

## 9. Human review

**DO: Open Review. Enter your reviewer label and rationale. Approve only if you agree and evidence is current; otherwise reject.**

Human review remains a separate decision. This creates a signed review record linked to the original result. The reviewer label is self-entered, and approval does not automatically apply or deploy the patch.

Example rationale, only if accurate: Reviewed the parameterized query change and the reported security, functional and mutation-check results. Approved for further integration testing.

## 10. Show the review record

**DO: Point to the decision and rationale, then click Open reviewed evidence.**

This record preserves what I decided and why, and links back to the original repair evidence.

In a real development workflow, a maintainer would apply the approved candidate to a development branch, run the project's full integration tests, and submit it through normal code review before merging and deployment.

Our demonstration covers live local AI generation, verification, inspectable changes, signed evidence and human review. Arbitrary-project support and production assurance require additional configuration and validation.

## If the demo is interrupted

Server unavailable: restart the normal launcher against the saved store and reopen the run. Slow generation: explain local inference and wait. Failed generation: show the actual failure; clearly label any older result as recorded. Stale evidence: explain why approval is blocked rather than presenting it as current.

Do not claim OWASP certification, universal repair capability, production deployment, or completion of the Azure qualification and campaign. These are outside this local demo.

## Final spoken line

An apply-to-branch or GitHub integration would be a useful future feature; it isn't implemented in the current demo.